

Plagiarism Report

Main File: Finalminiprojectreport.pdf

Average Similarity: 73%

Maximum Similarity: 100%

Comparison Results

1. FinalMiniprojectFrontpage.pdf

Similarity: 72%

2. java-pdf.pdf

Similarity: 48%

3. Finalminiprojectreport.pdf

Similarity: 100%

Main PDF Content

PDF PLAGIARISM CHECKER Dept. of CSE 2025 – 2026 P a g e | 1 CHAPTER 1: INTRODUCTION In today's digital world, the availability of online information and electronic documents has made content sharing easier than ever before. While this has improved access to knowledge and resources, it has also increased the risk of plagiarism in academic, research, and professional work. Students, researchers, and content creators often submit assignments, reports, and publications in PDF format, making it necessary to develop efficient systems capable of verifying the originality of such documents. Manual plagiarism checking is time- consuming, error-prone, and impractical for large volumes of data, thereby creating a strong need for automated plagiarism detection systems. Plagiarism refers to the act of copying or reproducing another person's work without proper acknowledgment. Existing plagiarism detection tools often focus mainly on text documents and may not effectively process PDF files, especially scanned PDFs containing image-based text. Furthermore, many traditional systems rely only on direct text matching, which makes them less effective in detecting paraphrased or semantically similar content. As a result, there is a growing demand for intelligent systems that can accurately analyze PDF documents and identify different forms of plagiarism. The proposed PDF Plagiarism Checker aims to overcome these limitations by integrating PDF text extraction, Optical Character Recognition (OCR), Natural Language Processing (NLP), and similarity analysis techniques into a single efficient system. The system allows users to upload PDF documents, after which the textual content is extracted automatically. OCR technology is used to retrieve text from scanned or image-based PDFs, ensuring compatibility with a wide range of document formats. Once the text is extracted, advanced NLP techniques such as TF-IDF and Cosine Similarity are applied to compare the document content with existing sources or datasets. The system is also capable of semantic similarity detection, enabling it to identify paraphrased or partially modified content rather than relying solely on exact word matching. Matching sections are highlighted, and a plagiarism percentage is generated to provide a clear understanding of the similarity level between documents.

PDF PLAGIARISM CHECKER Dept. of CSE 2025 – 2026 P a g e | 2 The system provides a user-friendly interface that simplifies the plagiarism checking process and generates detailed reports for users. By automating content analysis and comparison, the project improves accuracy, saves time, and promotes originality in digital document creation. The proposed solution is highly beneficial for educational institutions, research organizations, and professional environments where maintaining authenticity and ethical standards is essential. Through the integration of modern document processing and NLP techniques, the PDF Plagiarism Checker serves as an effective tool for detecting copied or similar content in PDF documents. It provides a scalable and reliable solution that addresses the growing need for intelligent plagiarism detection systems in the digital era.

PDF PLAGIARISM CHECKER Dept. of CSE 2025 – 2026 P a g e | 3 CHAPTER 2: SOFTWARE REQUIREMENTS To implement the proposed PDF Plagiarism Checker, several software tools and technologies are used to ensure efficient document processing, similarity detection, and report generation. Each tool plays an important role in achieving accurate plagiarism analysis and seamless system performance. The following software components are used in the project:

1. HTML HTML (HyperText Markup Language) is used to design the structure of the web pages in the system. It helps in creating the user interface for uploading PDF documents, displaying plagiarism results, and generating reports. HTML provides a simple and organized layout that allows users to interact easily with the plagiarism detection system.
2. CSS CSS (Cascading Style Sheets) is used for designing and styling the web interface of the application. It improves the visual appearance of the system by adding layouts, colors, fonts, and responsive design elements. CSS enhances the overall user experience by making the interface more attractive, readable, and user-friendly.
3. Node.js Node.js is used as the backend runtime environment for handling server-side operations in the system. It manages file uploads, processes requests from users, and connects different components of the application efficiently. Node.js enables fast and scalable execution of tasks such as PDF handling, text extraction, and similarity analysis.
4. PDF Parse PDF Parse is a library used for extracting textual content from PDF documents. It converts the uploaded PDF files into machine-readable text that can be analyzed further by the plagiarism detection algorithms. The tool supports efficient processing of digital PDF documents

and acts as a key component in the document analysis workflow. PDF PLAGIARISM CHECKER Dept. of CSE 2025 – 2026 P a g e | 4

5. OCR (Optical Character Recognition) OCR technology is used to extract text from scanned or image-based PDF files. Many plagiarism detection systems fail to process scanned documents effectively, but OCR enables this system to recognize and convert image text into editable and searchable content. This improves the accuracy and compatibility of the plagiarism checker for different types of PDF documents.

6. TF-IDF (Term Frequency–Inverse Document Frequency) TF-IDF is an NLP-based text analysis technique used to measure the importance of words in a document relative to other documents. It helps in identifying significant terms and converting textual data into numerical vectors for similarity comparison. TF- IDF plays a major role in detecting similarities between documents efficiently.

7. Cosine Similarity Cosine Similarity is a mathematical technique used to calculate the similarity between two text documents based on their vector representations. It determines how closely related two documents are by measuring the angle between their vectors. This method helps in generating plagiarism percentages and identifying copied or similar content accurately.

8. Jaccard Similarity Jaccard Similarity is another text comparison technique used to compare the similarity between two sets of words or phrases. It measures the common elements shared between documents and supports the plagiarism detection process by identifying overlapping content effectively.

9. MySQL MySQL is used as the database management system for storing user information, uploaded document details, plagiarism reports, and comparison results. It ensure efficient data management, retrieval, and storage within the application. MySQL provides reliability and scalability for handling multiple plagiarism analysis records.

PDF PLAGIARISM CHECKER Dept. of CSE 2025 – 2026 P a g e | 5

These software tools work together to create a reliable and efficient PDF Plagiarism Checker capable of processing documents, extracting text, analyzing similarities, and generating detailed plagiarism reports automatically. The integration of NLP techniques, OCR technology, and database management ensures accuracy, scalability, and ease of use in the system.

PDF PLAGIARISM CHECKER Dept. of CSE 2025 – 2026 P a g e | 6

CHAPTER 3: IMPLEMENTATION

Fig. 3.1 Design Diagram of our solution

PDF PLAGIARISM CHECKER Dept. of CSE 2025 – 2026 P a g e | 7

SOURCE CODE (sample) :

```
const { extractText } = require("../services/pdfService");
const { getChunks } = require("../services/chunkService");
const { similarity } = require("../services/similarityService");
const { searchWeb } = require("../services/searchService");
const { fetchPageText } = require("../services/webFetchService");
const { cosineSimilarity } = require("../tfidfSimilarityService");
const comparePDFs = async (req, res) => {
  try {
    const main = req.files["pdf1"]?.[0];
    const others = req.files["pdfs"];
    if (!main || !others) {
      return res.status(400).json({ error: "Missing PDFs" });
    }
    const mainText = await extractText(main.buffer);
    let results = [];
    for (let file of others) {
      const text = await extractText(file.buffer);
      const score = cosineSimilarity(mainText, text);
      results.push({ fileName: file.originalname, similarity: Math.round(score) });
    }
    const avg = results.reduce((sum, r) => sum + r.similarity, 0) / results.length;
    const max = Math.max(...results.map(r => r.similarity));
    res.json({ mainFile: main.originalname, percentage: Math.round(avg), maxSimilarity: Math.round(max), results });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};
const internetCheck = async (req, res) => {
  try {
    const file = req.file;
    if (!file) {
      return res.status(400).json({ error: "No PDF uploaded" });
    }
    const text = await extractText(file.buffer);
    const chunks = getChunks(text).slice(0, 5);
    console.log("Total chunks used:", chunks.length);
    let matches = [];
    PDF PLAGIARISM CHECKER Dept. of CSE 2025 – 2026 P a g e | 8
    let total = 0;
    const seenQueries = new Set();
    for (let chunk of chunks) {
      if (!chunk || chunk.length < 80) continue;
      const safeQuery = chunk .replace(/\s+/g, " ") .trim() .slice(0, 200);
      if (seenQueries.has(safeQuery)) continue;
      seenQueries.add(safeQuery);
      const results = await searchWeb(safeQuery);
      for (let r of results) {
        const url = r.url || r.link || r.href || "";
        if (!url || typeof url !== "string") continue;
        const pageText = await fetchPageText(url);
        if (!pageText || pageText.length < 100) continue;
        const score = similarity(chunk, pageText);
        if (score > 3) {
          matches.push({ chunk, source: url || "unknown-source", similarity: Math.round(score) });
          total += score;
        }
      }
    }
    const finalScore = matches.length === 0 ? 0 : total / matches.length;
    res.json({ mainFile: file.originalname, percentage: Math.round(finalScore), sources: matches });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: "Internet check failed" });
  }
};
module.exports = { comparePDFs, internetCheck };
Note: The above code is a part of the entire project and belongs to the file "plagiarismController.js".
```

PDF PLAGIARISM CHECKER Dept. of CSE 2025 – 2026 P a g e | 9

CHAPTER 4: RESULT

Fig. 4.1 Mode 1- PDF vs PDF's

Fig. 4.2 Mode 2- Internet Check

Fig. 4.3 Result of Mode 1

Fig. 4.4 Analyzing Internet Plagiarism

Fig. 4.5 Result of Mode 2

PDF

PLAGIARISM CHECKER Dept. of CSE 2025 – 2026 P a g e | 10 CHAPTER 5: CONCLUSION AND FUTURE SCOPE The PDF Plagiarism Checker provides an efficient solution for detecting plagiarism in digital PDF documents. With the increasing use of online resources and electronic submissions, maintaining originality has become very important. The system automates PDF text extraction, content comparison, and plagiarism analysis. The project uses technologies such as OCR, NLP-based semantic analysis, TF-IDF, Cosine Similarity, and Jaccard Similarity to detect both exact matches and paraphrased content accurately. It supports both normal and scanned PDF files, making it more effective than many traditional plagiarism detection systems. The system generates plagiarism percentages and highlights matching sections to provide clear and understandable reports. The use of HTML, CSS, Node.js, and MySQL ensures smooth performance, efficient data management, and a user-friendly interface. The automated workflow reduces manual effort, improves accuracy, and saves time during plagiarism verification. Overall, the project promotes academic integrity and demonstrates how NLP and document processing techniques can be used to build an intelligent and reliable plagiarism detection system. The PDF Plagiarism Checker can be further enhanced by integrating additional features and advanced technologies to improve its performance, scalability, and accuracy. Some possible future improvements include:

1. AI-Based Deep Semantic Analysis Advanced Artificial Intelligence and Machine Learning models can be integrated to improve semantic understanding and detect complex paraphrased plagiarism more accurately. This would allow the system to identify similarities based on meaning rather than only word matching.
2. Real-Time Web Crawling The system can be expanded to perform real-time comparison with online articles, journals, blogs, and research databases through web crawling techniques. This would increase the coverage and effectiveness of plagiarism detection.
3. Multi-Language Support Future versions of the system can support plagiarism detection in multiple languages, making it more useful for international users and multilingual academic environments.
4. Cloud-Based Storage and Processing Migrating the system to cloud platforms can improve scalability, accessibility, and storage management. Cloud integration would also enable users to access plagiarism reports from anywhere in real time.
5. Improved OCR Accuracy Enhancing OCR capabilities with advanced image processing techniques can improve text extraction from low-quality scanned PDFs and handwritten documents.
6. Detailed Report Generation The system can be upgraded to generate more detailed plagiarism reports, including source references, similarity summaries, graphical analysis, and downloadable reports in PDF format.
7. Integration with Educational Platforms The plagiarism checker can be integrated with Learning Management Systems (LMS) such as Moodle or Google Classroom to allow direct plagiarism verification of assignments and research submissions.
8. User Authentication and Role Management Adding secure login systems with role-based access for students, teachers, and administrators can improve system security and usability.

PDF PLAGIARISM CHECKER Dept. of CSE 2025 – 2026 P a g e | 12

9. Mobile Application Support A mobile application version of the plagiarism checker can be developed to provide easy access and quick plagiarism verification using smartphones and tablets. By implementing these future enhancements, the PDF Plagiarism Checker can evolve into a more intelligent, scalable, and comprehensive plagiarism detection platform capable of meeting future academic and professional requirements effectively.

PDF PLAGIARISM CHECKER Dept. of CSE 2025 – 2026 P a g e | 13

REFERENCES %İ <https://youtu.be/D2V1okCEsIE?si=mg7FqTMxQgDq06OJ> %İ www.tavily.com %İ <https://github.com/topics/plagiarism-checker> %İ <https://github.com/scikit-learn/scikit-learn>